ADVANCED PROGRAMMING LANGUAGES

Spring 2026, CM 0632. MSc in Computer Science and Information Technology, Ca' Foscari University of Venice

Designing higher-order functions

In the previous lectures we have practiced with various uses of higher-order functions. Now it's time to look at how higher-order functions can be defined.

Higher-order collection processing

We start with the design of higher-order operations on collections, taking list processing as a motivating context. As we have seen, several patterns emerge over and over:

- transforming every element of a list,
- verifying whether a property holds for all the elements of a list,
- extracting from a list elements satisfying a certain condition,
- combining the elements of a list
- . . . and more.

In imperative languages, the above patterns would be expressed by idiomatic combinations of for or while loops. In functional languages they are expressed more concisely with higher-order functions. We discuss the design of these patterns on lists as one concrete example: the same design applies, mutatis mutandis, to the other concrete collections.

Polymorphic functions and function types. One distinguishing, very common feature of higher-order functions is that they have polymorphic function types — types that include type variables standing for arbitrary types. For the time being we rely on intuition: we'll return to polymorphism and polymorphic function types later on.

A note on syntax: Haskell vs Scala. Throughout these lectures, we present higher-order functions primarily in Haskell. The same definitions can be written in Scala, with one visible difference: in Haskell, type variables appear only in the *type signature*, and the function definition is free of type annotations; in Scala, type parameters must be declared explicitly in the `def`, alongside the code. Compare:

```haskell
-- Haskell: type variables in the signature, clean definition
map :: (a -> b) -> [a] -> [b]
map f []     = []
map f (x:xs) = f x : map f xs
```

```scala
// Scala: type parameters [A,B] in the def, types annotated inline
def map[A,B](f: A => B, xs: List[A]): List[B] = xs match
  case Nil    => Nil
  case x::xs  => f(x) :: map(f, xs)
```

This difference reflects a deeper distinction in the type inference engines of the two languages (Hindley–Milner in Haskell vs local type inference in Scala), which we will discuss in detail later. For now, it suffices to note that the two definitions are expressing the same function — and that the Haskell version is more concise precisely because the type checker infers more.

In Haskell, functions are also *curried by default*: a function of two arguments like `map` is really a function that takes one argument and returns a function expecting the second. This means that partial application — supplying fewer arguments than expected — is free:

```haskell
double = map (* 2)    -- a function [Int] -> [Int], no extra syntax needed
```

In Scala, achieving the same effect requires an explicitly curried signature or the `.curried` method. We will return to currying in Part II.

With that understood, we proceed in Haskell.

Mapping

We start with `map` and `concatMap`.

- `map` takes a function `f` and a list `xs` and returns the list obtained by applying `f` to each element of `xs`.
- `concatMap` takes a list-valued function `f` and a list `xs`, maps `f` over `xs`, and concatenates the resulting lists into a *flat* list.

```haskell
map :: (a -> b) -> [a] -> [b]
map f []     = []
map f (x:xs) = f x : map f xs

concatMap :: (a -> [b]) -> [a] -> [b]
concatMap f []     = []
concatMap f (x:xs) = f x ++ concatMap f xs
```

Some examples:

```haskell
map (+1) [1,2,3,4,5]           -- [2,3,4,5,6]
map length ["a","word","list"]  -- [1,4,4]

concatMap (\w -> [w, w]) [1,2,3]  -- [1,1,2,2,3,3]
```

Note how `map` preserves the list structure (the output has the same length as the input), while `concatMap` may change it. In Scala, these two functions are called `map` and `flatMap`.

Filtering

Given a predicate (a Boolean function) `p` and a list `xs`, `filter` returns the list of those elements of `xs` that satisfy `p`:

```haskell
filter :: (a -> Bool) -> [a] -> [a]
filter p []     = []
filter p (x:xs)
  | p x       = x : filter p xs
  | otherwise  = filter p xs


filter even [1,2,3,4,5]               -- [2,4]
filter (\w -> length w > 2) ["a","word","list"]  -- ["word","list"]
```

Folding

Another common kind of operation combines the elements of a list with the same operator. For instance:

```haskell
sum :: [Int] -> Int
sum []     = 0
sum (x:xs) = x + sum xs

prod :: [Int] -> Int
prod []     = 1
prod (x:xs) = x * prod xs
```

We see the common pattern emerging. Specifically:

- `sum [a1,...,an] = f a1 (f a2 (... (f an b) ...))` where `f = (+)` and `b = 0`
- `prod [a1,...,an] = f a1 (f a2 (... (f an b) ...))` where `f = (*)` and `b = 1`

We may abstract over the common pattern and define the recursion scheme parametrically on the function to be applied and on the value to be returned when the list is empty. This operation is traditionally called `foldr` (*fold right*):

```haskell
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b []     = b
foldr f b (a:as) = f a (foldr f b as)
```

Folding is a rather general recursion scheme, which may serve a variety of purposes depending on the function we pass in. Consider the invocation `foldr f b as`:

- if `as` is the empty list, we simply return `b`;
- when the list is `a:as`, from the structure of the recursive call `f a (foldr f b as)` it follows that `f` receives as its first argument the head `a` and as its second argument the result of folding over the tail. The body of `f` must be defined accordingly.

Keeping that in mind, `sum` and `prod` are readily redefined:

```haskell
sum  = foldr (+) 0
prod = foldr (*) 1
```

Notice how clean partial application makes these definitions: `foldr (+) 0` is itself a function `[Int] -> Int`.

Indeed, most recursive functions on lists can be encoded as instances of folding:

```haskell
length :: [a] -> Int
length = foldr (\_ n -> n + 1) 0

append :: [a] -> [a] -> [a]
append xs ys = foldr (:) ys xs

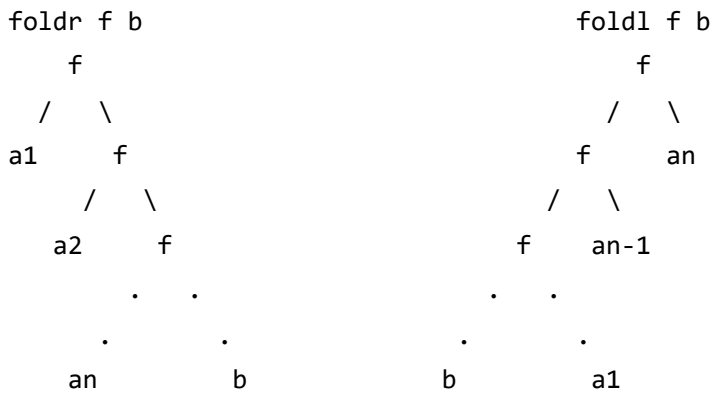
reverse :: [a] -> [a]
reverse = foldr (\x acc -> acc ++ [x]) []
```

Even `map` and `filter`:

```haskell
map f    = foldr (\x acc -> f x : acc) []
filter p = foldr (\x acc -> if p x then x : acc else acc) []
```

FoldRight vs FoldLeft

We already observed that `foldr f b [a1,...,an]` implements the composition scheme `f a1 (f a2 (... (f an b) ...))`. Graphically, this can be rendered as a right-leaning tree. Perhaps unsurprisingly, there is a dual folding scheme, `foldl`, that produces left-leaning trees and corresponds to the composition scheme `f (... (f (f b a1) a2) ...) an`:



The `foldl` definition is:

```haskell
foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b []     = b
foldl f b (a:as) = foldl f (f b a) as
```

Like `foldr`, `foldl` implements the iterated application of `f` to combine the elements of the list, but the accumulation works differently. Consider `foldl f b (a:as)`:

- the second argument of `foldl` acts as an **accumulator**: `b` is the initial value, and `f b a` extends it by one step before continuing with the tail `as`.

For associative operations, `foldl` and `foldr` produce the same result, but they differ in efficiency. Crucially, `foldl` is **tail-recursive**: the recursive call is the last operation, so it can execute in constant stack space. `foldr`, by contrast, builds up a chain of deferred applications proportional to the list length, and will overflow the stack on very long lists. (Haskell's `Data.List` provides a strict variant `foldl'` that is the preferred choice in practice.)

```haskell
sum  = foldl (+) 0
prod = foldl (*) 1

length :: [a] -> Int
length = foldl (\n _ -> n + 1) 0

reverse :: [a] -> [a]
reverse = foldl (\acc x -> x : acc) []
```

Notice how `reverse` becomes particularly natural with `foldl`: consing each element onto the accumulator produces the reversed list, with no concatenation needed.

Scala note. In Scala's collections, `foldRight` and `foldLeft` are methods on lists. Their signatures are curried, with the initial value and the combining function supplied in separate argument groups:

```scala
xs.foldRight(b)((a, acc) => ...)
xs.foldLeft(b)((acc, a) => ...)
```

Reduce.
Haskell provides `foldr1` and `foldl1`, which fold a non-empty list using its first (or last) element as the initial value, without requiring an explicit base:

```haskell
foldr1 :: (a -> a -> a) -> [a] -> a
foldr1 f [x]    = x
foldr1 f (x:xs) = f x (foldr1 f xs)

sum  = foldl1 (+)    -- only on non-empty lists
prod = foldl1 (*)
```

In Scala, the analogous operation is called `reduce`. An important conceptual point: `reduce` (and `foldr1` / `foldl1`) requires its operator to be of type `a -> a -> a` — both arguments and the result

must share the same type. This is more restrictive than `foldr` / `foldl`, where the accumulator may have a different type from the list elements. The trade-off is that, precisely because the types coincide, a `reduce` over an associative and commutative operation can in principle be **parallelized**: the list can be split into chunks, each chunk reduced independently, and the partial results combined in any order. This is the foundation of the map-reduce paradigm in distributed computing. A `fold`, by contrast, is inherently sequential because the accumulation proceeds left-to-right or right-to-left.

REFERENCES

The material of this lecture draws on Chapter 5 of *Programming in Haskell (2nd Edition)*, G. Hutton, Cambridge University Press, 2016, and Chapter 15 of *Programming in Scala (5th Edition)*, M. Odersky, L. Spoon, B. Venners, F. Sommers, Artima Press, 2021.